Principle of Programming Language

Sessional Exam - Detailed Solutions

Q-1. Briefly explain programming language history and its characteristics.

Part 1: History of Programming Languages

The evolution of programming languages can be categorized into distinct phases or generations:

- **1950s (The Dawn of High-Level Languages):** Programming moved away from pure machine code and assembly.
  - *FORTRAN (1957):* Developed by IBM, it was the first widely used high-level language, designed for scientific and engineering computations.
  - *LISP (1958):* Created for artificial intelligence research, introducing functional programming.
  - *COBOL (1959):* Designed specifically for business data processing.
- **1960s - 1970s (Structured Programming & System Languages):**
  - *ALGOL (1960):* Introduced block structure and lexical scoping.
  - *C (1972):* Developed at Bell Labs by Dennis Ritchie. It provided low-level memory access with high-level syntax, becoming the standard for system programming (like Unix).
- **1980s - 1990s (The Rise of Object-Oriented Programming):**
  - *C++ (1983):* Added object-oriented features (classes, inheritance) to C.
  - *Java (1995):* Designed by Sun Microsystems with the "Write Once, Run Anywhere" philosophy, heavily relying on the Java Virtual Machine (JVM).
  - *Python (1991):* Emphasized code readability and rapid development.
- **2000s - Present (Modern & Web-Centric Languages):**
  - Languages like C#, JavaScript, Swift, and Rust emerged to handle web development, mobile apps, and memory safety.

Part 2: Characteristics of a Good Programming Language

A good programming language should possess the following key characteristics:

1. **Readability:** The code should be easy to read and understand. This makes maintenance and debugging easier.

2. **Writability:** The language should allow programmers to express complex ideas easily and concisely.
3. **Reliability:** A language is reliable if it performs to its specifications under all conditions. Features like strict type checking and exception handling contribute to this.
4. **Orthogonality:** A relatively small set of primitive constructs can be combined in a relatively small number of ways to build the control and data structures of the language. (e.g., every instruction should have a single, distinct purpose).
5. **Portability:** Programs written in the language should be easily transferable from one computer system to another (like Java's bytecode).
6. **Efficiency:** The language should allow for the generation of fast, optimized executable code and use memory efficiently.

Q-2. Describe programming language translators and also show the comparison between them.

Programming Language Translators

Computers only understand machine language (binary 0s and 1s). Translators are system software programs that convert high-level source code (written by humans) into machine-level code (executable by hardware).

There are three main types of translators:

1. **Assembler:** Converts assembly language (low-level, mnemonic-based) into machine code. It is highly hardware-dependent.
2. **Compiler:** Translates the entire high-level source program into machine code in a single batch before execution. If there are syntax errors, it lists them all at once and refuses to generate the executable file until they are fixed.
3. **Interpreter:** Translates high-level source code into machine code line-by-line during execution. It reads a line, translates it, and executes it immediately before moving to the next.

Comparison: Compiler vs. Interpreter

Feature	Compiler	Interpreter
Input	Scans the entire program at once.	Scans the program line-by-line.
Output	Generates an intermediate object code (.obj) or executable file (.exe).	Does not generate an intermediate object code; executes directly.
Execution Time	Execution is much faster because the code is already compiled.	Execution is slower due to the simultaneous translation and execution overhead.
Memory Requirement	Requires more memory (needs space for the source code, compiler, and generated object code).	Requires less memory (no object code is generated or stored).
Error Detection	Displays all errors after scanning the whole program. Debugging is comparatively harder.	Stops execution upon finding the first error. Debugging is easier and faster.
Examples	C, C++, Go, Rust.	Python, Ruby, JavaScript, PHP.

Q-3. What are structured data objects and data types? Also explain its specification and implementation.

Structured Data Objects and Data Types

- **Data Object:** A run-time grouping of one or more pieces of data in a virtual computer.
- **Structured Data Type:** Also known as composite data types, these are user-defined or built-in types that group multiple elementary (primitive) data items or other structured items together. They allow programmers to model complex real-world entities.
    - *Examples:* Arrays (homogeneous elements), Records/Structs (heterogeneous elements), Sets, and Files.

1. Specification of Structured Data Types

Specification defines the logical properties of the data structure, independent of how it is stored in memory. It includes:

- **Number of Components:** Is the size fixed (like a standard array) or variable (like a linked list)?
- **Type of Each Component:** Are all elements of the same type (homogeneous, e.g., an array of integers) or different types (heterogeneous, e.g., a struct containing an int, string, and float)?
- **Names to Select Components:** How are individual elements accessed? (e.g., using integer indices `A[i]` for arrays, or field names `student.age` for records).
- **Maximum Number of Components:** The upper bound on the size of the structure.
- **Operations:** What operations can be performed? (e.g., inserting, deleting, sorting, merging).

2. Implementation of Structured Data Types

Implementation deals with how the structured data is actually stored in computer memory and how the specified operations are executed.

- **Storage Representation:**
    - *Sequential Representation (Contiguous Mapping):* Elements are stored in adjacent memory locations. (e.g., Arrays). This allows for fast, random access (O(1)) but makes insertion/deletion difficult.
    - *Linked Representation:* Elements are scattered in memory, but each element contains a pointer (memory address) to the next element. (e.g., Linked Lists). This makes insertion/deletion easy but requires sequential access.

- **Implementation of Operations:** Writing the actual code/algorithms (like loops and pointer arithmetic) that manipulate the memory layout to perform tasks like selecting a specific record field or indexing an array.

Q-4. Define elementary data types and also the implementation of elementary data types.

Elementary Data Types

Elementary (or primitive) data types are the most basic, fundamental built-in types provided by a programming language. They cannot be broken down into simpler constituent data types. They hold a single, simple value.

- **Common Examples:** Integer, Float (real numbers), Character, and Boolean.

Implementation of Elementary Data Types

Implementation refers to how these logical types are represented in the hardware's physical memory (RAM).

1. **Integer Implementation:**
    - Represented using fixed-length blocks of bits (e.g., 16-bit, 32-bit, or 64-bit words).
    - Negative numbers are almost universally implemented using **Two's Complement** notation, which simplifies the hardware design for ALUs (Arithmetic Logic Units).
    - *Example:* An 8-bit integer can store values from -128 to 127.
2. **Floating-Point (Real Number) Implementation:**
    - Implemented based on the **IEEE 754 Standard**.
    - Memory is divided into three parts:
        - **Sign bit:** 1 bit (0 for positive, 1 for negative).
        - **Exponent:** Represents the power of the base (usually 8 bits for single precision).
        - **Mantissa (or Significand):** The fractional part of the number (usually 23 bits for single precision).
3. **Character Implementation:**
    - Characters are implemented using standardized numeric codes. The hardware stores a binary number, and the software translates it into a human-readable glyph.
    - *ASCII:* Uses 8 bits (1 byte) per character, sufficient for basic English (e.g., 'A' is stored as binary 01000001, which is decimal 65).

- *Unicode (UTF-8/UTF-16):* Uses 16 to 32 bits to represent characters from almost all global languages and emojis.

4. **Boolean Implementation:**
   - Logically requires only a single bit (0 for False, 1 for True).
   - However, because computer memory is byte-addressable (the smallest chunk of memory you can easily access is 1 byte), booleans are typically implemented using an entire byte (8 bits) where `00000000` is False and `00000001` (or any non-zero value) is True.

Q-5. Define syntax and explain formal method of describing syntax.

Definition of Syntax

In programming languages, **syntax** refers to the set of formal rules that dictate how the symbols, keywords, and characters of a language must be combined to form valid statements and programs.

- It is the grammar of the language.
- *Note:* Syntax only checks if the structure is correct, not if the code makes logical sense (that is *semantics*). For example, `x = a + b;` is syntactically correct in C, but if `a` is a string and `b` is an integer, it is semantically incorrect.

Formal Methods of Describing Syntax

Because programming languages must be compiled by machines, their syntax must be defined mathematically and unambiguously. The most common formal methods are:

1. Backus-Naur Form (BNF)

BNF is a meta-language (a language used to describe another language). It uses rules or "productions" to define syntactic structures.

- **Components of BNF:**
  - **Non-terminals:** Concepts being defined, enclosed in angle brackets `< >`. (e.g., `<assignment_stmt>`).
  - **Terminals:** The actual literal characters/keywords of the language (e.g., `if`, `while`, `=`, `+`).
  - **Metasymbols:** `::=` means "is defined as", and `|` means "or".
- *Example of an Assignment Statement in BNF:*
  `<assignment_stmt> ::= <identifier> = <expression>`

`<identifier> ::= a | b | c`
`<expression> ::= <identifier> + <identifier> | <identifier>`

2. Extended BNF (EBNF)

EBNF adds special meta-symbols to standard BNF to make the rules shorter and easier to read.

- `[ ]` (Optional): Everything inside the brackets is optional.
- `{ }` (Repetition): Everything inside the braces can be repeated zero or more times.
- `( )` (Grouping): Used to group elements together.
- *Example (If statement in EBNF):*
  `<if_stmt> ::= if ( <condition> ) <statement> [ else <statement> ]`

3. Syntax Graphs (Railroad Diagrams)

This is a visual, graphical representation of EBNF rules.

- Non-terminals are represented by rectangles.
- Terminals are represented by circles or ovals.
- Directed arrows (lines) connect them to show the flow/sequence of valid combinations.
- Loops in the arrows represent repetition (`{ }`), and bypass routes represent optional elements (`[ ]`).